

Low-Discrepancy Sampling on Higher-Dimensional Spheres

Wai-Shing Luk

Fudan University

September 25, 2026



- 1 Motivation
- 2 Overview of Low-Discrepancy Sequences
- 3 Our Approach
- 4 Numerical Experiments
- 5 Conclusions

Section 1

Motivation

Low-Discrepancy Sequences

- A **deterministic** sequence of points that fills a space more uniformly than random sampling
- Three desirable properties:
 - **Uniformity** – points are spread evenly, minimising clusters and gaps
 - **Determinism** – the sequence is reproducible from a fixed starting point
 - **Incrementality** (*the key property*) – points can be appended while the whole set stays evenly distributed
- Drawback: generation becomes slower as the number of points grows

Applications of LDS on S^n

- **Robot motion planning** on S^3 and $SO(3)$ (Yershova et al. 2010) – for path planning and attitude control
- **Wireless coding** – spherical codes and MIMO codebooks (Utkovski and Lindner 2006)
- **Multivariate empirical mode decomposition** (Rehman and Mandic 2010) – more accurate signal models
- **Filter bank design** (Mandic et al. 2011) – more precise filter parameters
- **Statistical and machine learning** – deterministic coverage of a normalised parameter space

The Challenge in Higher Dimensions

- Sampling the 3-D sphere ($n = 2$) is well understood
- Little is known for $n > 2$ – the *curse of dimensionality*
- The **cylindrical mapping does not generalise** to higher dimensions
- Goal: one construction on S^n that is uniform, deterministic and incremental

Section 2

Overview of Low-Discrepancy Sequences

The van der Corput Sequence

- The one-dimensional low-discrepancy sequence on $[0, 1]$
- Constructed by reversing the base- b digits of the integers, with b usually prime
- Introduced by the Dutch mathematician **Johannes van der Corput** (1935)



Unit Circle S^1

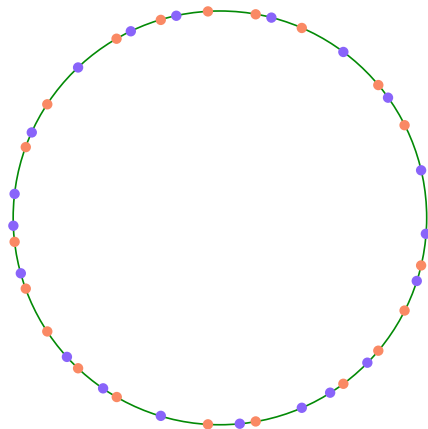
- Treat the vdc value as an angle:

$$\theta = 2\pi \cdot \text{vdc}(k, b)$$



$$[x, y] = [\cos \theta, \sin \theta]$$

- Incremental and uniform by construction

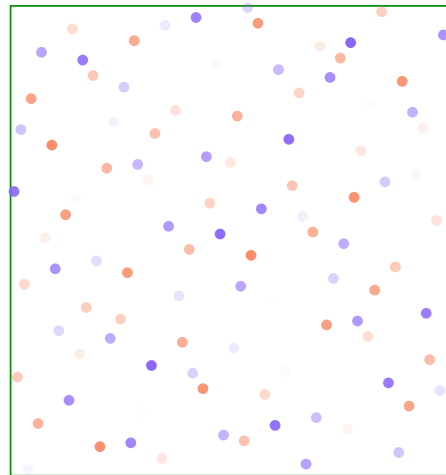


Halton Sequence on $[0, 1]^n$

- Combine n van der Corput sequences with **coprime** bases:

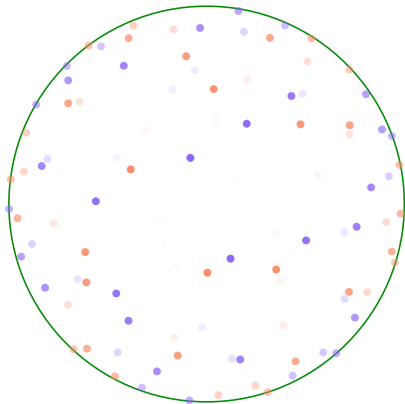
$$[x_1, \dots, x_n] = [\text{vdc}(k, b_1), \dots, \text{vdc}(k, b_n)]$$

- Uniform over the unit hypercube
- Basis of quasi-Monte Carlo (QMC) methods



Unit Sphere S^2 – Cylindrical Mapping

- Two vdc values give z and φ :
 - $z = 2 \text{vdc}(k, b_2) - 1 \in [-1, 1]$
 - $\varphi = 2\pi \text{vdc}(k, b_1) \in [0, 2\pi)$
- Radius of the horizontal circle:
 $r = \sqrt{1 - z^2}$
- $$[x, y, z] = [r \cos \varphi, r \sin \varphi, z]$$
- Used in computer graphics (Wong et al. 1997)



S^3 and $SO(3)$ – Hopf Fibration

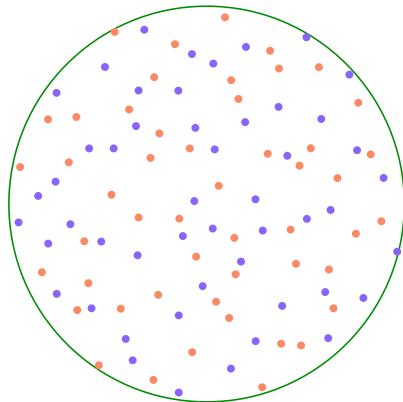
- Hopf coordinates (Yershova et al. 2010):
 - $x_1 = \cos(\theta/2) \cos(\psi/2)$
 - $x_2 = \cos(\theta/2) \sin(\psi/2)$
 - $x_3 = \sin(\theta/2) \cos(\varphi + \psi/2)$
 - $x_4 = \sin(\theta/2) \sin(\varphi + \psi/2)$
- S^3 is a principal circle bundle over S^2
- Works for S^3 only: the cylindrical mapping does **not** extend to S^n for $n > 2$

Section 3

Our Approach

Uniform Sampling on a Unit Disk

- Surface element: $dA = r dr d\theta$
- Integrating gives $\int r dr = \frac{1}{2}r^2$, so the radial weight is r
- Cancel it with the **inverse**: use
$$r = \sqrt{\text{vdc}(k, b_2)}$$
- $\theta = 2\pi \text{vdc}(k, b_1)$
- $[x, y] = [r \cos \theta, r \sin \theta]$



Recursive Construction on S^n

- Polar coordinates:
 - $x_0 = \cos \theta_n$
 - $x_1 = \sin \theta_n \cos \theta_{n-1}$
 - ...
 - $x_n = \sin \theta_n \sin \theta_{n-1} \cdots \sin \theta_1$
- The surface element factorises, but its **inverse has no closed form** for $n \geq 2$
- Key idea:

$$p_n = [\cos \theta_n, \sin \theta_n \cdot p_{n-1}]$$

building S^n from a point on S^{n-1}

How to Generate the Point Set

- Let $f_j(\theta) = \int \sin^j \theta \, d\theta$, defined recursively:

$$f_j(\theta) = \begin{cases} \theta & j = 0, \\ -\cos \theta & j = 1, \\ \frac{1}{n}(-\cos \theta \sin^{j-1} \theta + (n-1)f_{j-2}(\theta)) & j \geq 2. \end{cases}$$

- f_j is monotone on $(0, \pi)$; map $\text{vdc}(k, b_j)$ onto $[f_j(0), f_j(\pi)]$
- Invert numerically: $\theta_j = f_j^{-1}(t_j)$ by **table lookup**
- Then recurse with $p_n = [\cos \theta_n, \sin \theta_n \cdot p_{n-1}]$

Implementation

- SphereGen – abstract interface (pop, reseed)
- Sphere3 – explicit generator for S^3
- SphereN – recursive chain that bottoms out at Sphere3
- Inverse tables are built once by linear interpolation and **cached**

Reference Implementations

- Four sibling projects, all implementing the same generators:
 - `lds-gen` (Python) – `locks`, `pop_batch()`
 - `lds-rs` (Rust) – `AtomicU64`, `SphereN`
 - `lds-cpp` (C++20) – `constexpr`, precomputed tables
 - `lds-gen-cpp` (C++20) – includes the `sphere_n` module
- Core generators in all four: `VdCorput`, `Halton`, `Circle`, `Disk`, `Sphere`, `Sphere3Hopf`, `HaltonN`
- The recursive `Sphere3/SphereN` exist in Python, Rust and `lds-gen-cpp` only

Cross-Language Verification and Performance

- With a fixed seed the core generators are **bit-identical** across languages
- The recursive sphere generators agree to 15+ decimal places; one coordinate differed at the 16th (machine epsilon)
- Nanoseconds per point:
 - Sphere3 (S^3): C++ 657, Rust 1035, Python 7749
 - SphereN (S^4): Rust 1607, C++ 1665, Python 12153
 - SphereN (S^5): Rust 2020, C++ 2312, Python 48443
- C++ templates win for shallow generators; Rust's lock-free counter wins for deep recursion

Section 4

Numerical Experiments

Dispersion Measure

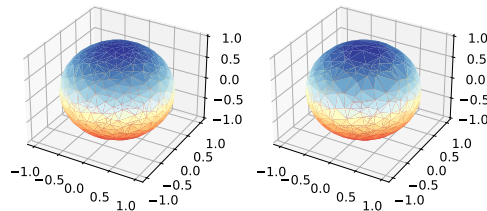
- Generate N points and build their convex hull (`scipy.spatial.ConvexHull`)
- Dispersion is the spread of neighbour distances:

$$\max_{a \in \mathcal{N}(b)} \{D(a, b)\} - \min_{a \in \mathcal{N}(b)} \{D(a, b)\}, \quad D(a, b) = \sqrt{1 - a^\top b}$$

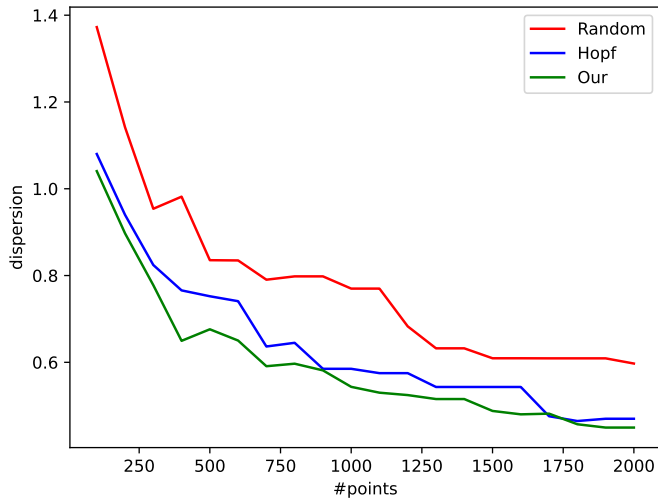
- Lower dispersion means a more uniform point set

Random vs LDS ($N = 600$)

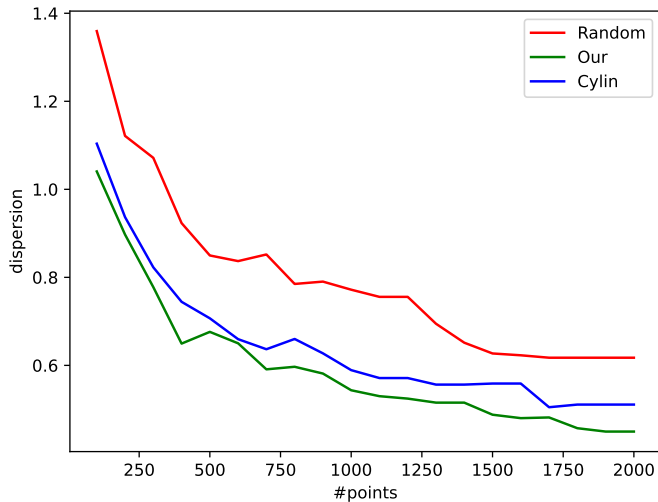
- **Random**: normalise a Gaussian vector, which is uniform on S^n
- **LDS**: the recursive generator
- Left: ours, right: random



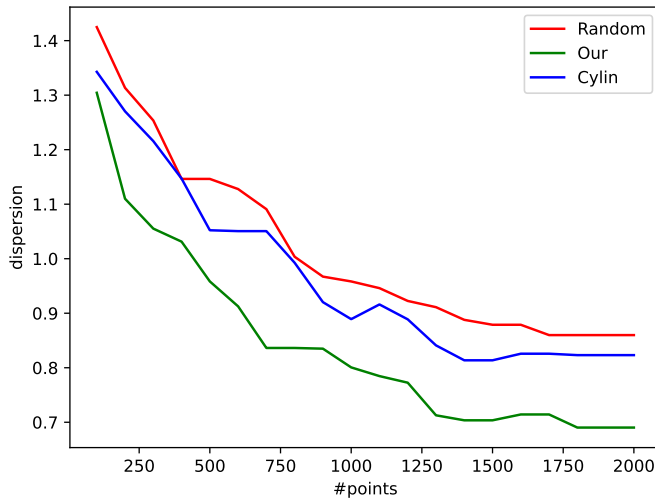
Results: S^3 vs Hopf Coordinate Method



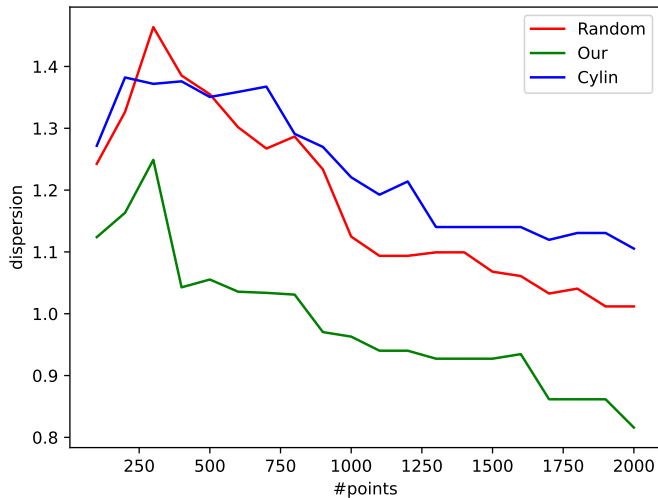
Results: S^3 vs Cylindrical Mapping



Results: S^4 vs Cylindrical Mapping



Results: S^5 vs Cylindrical Mapping



Section 5

Conclusions

Conclusions

- A uniform, deterministic and **incremental** LDS on S^n
- Built from the van der Corput sequence plus a recursive spherical construction
- Outperforms random sampling, especially when the number of points is small
- Superior to the Hopf method on S^3 and to cylindrical mapping on S^4 and S^5

Future Work

- Monte Carlo integration, optimisation and machine learning
- Faster table lookup or closed-form approximations for large n
- Extensions to $SO(n)$ and other manifolds

References I

Mandic, DP et al. 2011. “Filter Bank Property of Multivariate Empirical Mode Decomposition.” *Signal Processing, IEEE Transactions on* 59 (5): 2421–26.

Rehman, Naveed, and Danilo P Mandic. 2010. “Multivariate Empirical Mode Decomposition.” *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Science* 466 (2117): 1291–302.

Utkovski, Zoran, and Juergen Lindner. 2006. “On the Construction of Non-Coherent Space Time Codes from High-Dimensional Spherical Codes.” *Spread Spectrum Techniques and Applications, 2006 IEEE Ninth International Symposium on*, 327–31.

References II

Wong, Tien-Tsin, Wai-Shing Luk, and Pheng-Ann Heng. 1997. “Sampling with Hammersley and Halton Points.” *Journal of Graphics Tools* 2 (2): 9–24.

Yershova, Anna, Swati Jain, Steven M LaValle, and Julie C Mitchell. 2010. “Generating Uniform Incremental Grids on $SO(3)$ Using the Hopf Fibration.” *The International Journal of Robotics Research* 29 (7): 801–12.